



AI-Powered Plant Disease Detection with Generative AI

24.12.2025

R.Sahaana

Project Objective

Objective Develop a computer vision and Generative AI system that detects plant diseases from leaf images and provides simple, farmer-friendly explanations and treatment recommendations, optionally translated into a language chosen by the user. The system must combine Teachable Machine for image classification and LLM-based explanations (ChatGPT/Gemini/Claude) to help farmers understand the diagnosis and take actionable steps.

Problem Statement

Plant diseases are difficult to identify at an early stage, especially for farmers without access to expert guidance. Misdiagnosis or delayed treatment can lead to significant crop loss. There is a need for an easy-to-use AI system that can detect plant diseases from leaf images and provide simple, actionable treatment recommendations in a farmer-friendly manner.

Datasets

The dataset was provided as a ZIP file named **Plant_leave_diseases_dataset.zip**, containing labeled plant leaf images belonging to **six different classes**.

After extraction, the dataset was organized into two main folders:

- **Training folder** – Used to train the image classification model in Google Teachable Machine.

- **Testing folder** – Used to validate and test the trained model's performance.

The dataset includes the following six plant health classes:

- **Apple_Scab**
- **Apple_Healthy**
- **Potato_Early_Blight**
- **Potato_Healthy**
- **Tomato_Leaf_Mold**
- **Tomato_Healthy**

TOOLS USED:

Teachable Machine – Used to train an image classification model by uploading and labeling plant leaf images without complex coding.

Google Colab – Used as a cloud-based environment to integrate the trained model, test predictions, and make LLM API calls.

TensorFlow / Keras – Used as the backend framework to load the trained model and perform image-based disease prediction.

Gradio – Used to create a simple web interface that allows users to upload plant leaf images and view AI-generated explanations.

LLM Platform (OpenAI / Gemini / Claude) – Used to generate farmer-friendly explanations, remedies, and preventive guidance based on the model's prediction.

Dataset – A labeled plant disease image dataset containing six classes of healthy and diseased leaf images used for training and testing the model.

Solution Approach:

The solution follows a step-by-step approach to detect plant diseases and generate meaningful explanations. First, a plant leaf image classification model is trained using Google Teachable Machine with labeled training images. The trained model is then exported and integrated into Google Colab using TensorFlow.

When a user uploads a leaf image, the system preprocesses the image and uses the trained model to predict whether the plant is healthy or affected by a disease. Based on the prediction, a Large Language Model (Gemini/OpenAI/Claude) is invoked to generate simple, farmer-friendly explanations, treatment recommendations, and preventive measures. Finally, the results are displayed through an interactive Gradio-based web interface, allowing users to easily upload images and understand the diagnosis.

IMPLEMENTATION STEPS:

Step 1: Dataset Preparation

The first step involved preparing the plant disease dataset provided for the project. The dataset, named *Plant_leave_diseases_dataset.zip*, was downloaded and extracted. It contains images of plant leaves categorized into six different classes representing healthy and diseased conditions.

The dataset is organized into two main folders: a **Training folder**, which is used for model training in Teachable Machine, and a **Testing folder**, which is used later to evaluate the performance of the trained model. Each folder contains subfolders corresponding to the following class labels:

- Apple_Scab
- Apple_Healthy
- Potato_Early_Blight
- Potato_Healthy

- Tomato_Leaf_Mold
- Tomato_Healthy

All images were verified to ensure correct labeling and adequate representation in each class. Proper dataset structuring is essential to ensure accurate learning and reliable disease identification by the model.

Step 2: Training the Plant Disease Classification Model using Teachable Machine

The plant disease classification model was trained using **Google Teachable Machine**, an intuitive, no-code platform for building machine learning models. An image classification project was created by selecting *Get Started* → *Image Project* → *Standard Image Model*.

Six class labels were created corresponding to the dataset categories. Images from the Training folder were uploaded into their respective class labels. Training parameters were configured to improve model accuracy, with the number of epochs set to 200, batch size set to 16, and learning rate set to 0.001.

After configuring the parameters, the model was trained using the built-in training option. Test images were then uploaded into the preview section to verify prediction accuracy. If necessary, the model was retrained by adjusting parameters or adding more images.

Once satisfactory performance was achieved, the trained model was exported in **TensorFlow SavedModel** format and downloaded as a ZIP file. The Teachable Machine project was also saved to Google Drive to ensure backup and reproducibility. This step enabled the creation of a convolutional neural network capable of identifying plant diseases from leaf images.

Step 3: Model Integration in Google Colab

The exported TensorFlow SavedModel was integrated into **Google Colab** for further processing and deployment. The provided project notebook was opened in Colab, and the model ZIP file was uploaded.

Within the notebook, the model was loaded and tested using TensorFlow. Input images were resized and normalized to match the model's expected input format. Predictions were

performed on images from the Testing folder, and the output probabilities were mapped to the six disease classes.

The predicted class label obtained from the model was stored and passed as input to the Large Language Model (LLM) for explanation generation. Google Colab was used because it provides a cloud-based environment with computational resources and seamless integration with AI libraries.

Step 4: Generating Explanations, Treatments, and Multilingual Output using LLMs

To provide meaningful and farmer-friendly guidance, a Large Language Model (OpenAI, Gemini, or Claude) was integrated into the Colab notebook. The respective API client was imported, and a structured prompt was designed.

The prompt included the predicted disease label, optional confidence score, and requests for a simple explanation, treatment recommendations, preventive measures, and output in the user's preferred language. The prompt instructed the LLM to behave as an agricultural expert and provide practical, safe, and easy-to-follow remedies.

The LLM response was generated based on the prediction result. If the plant was unhealthy, the output included the plant name, disease name, symptoms, immediate actions, treatment options, and preventive steps. If the plant was healthy, the output confirmed the healthy status. This step ensured the system produced actionable and multilingual AI-generated guidance.

Step 5: Development of Gradio-Based User Interface

A user-friendly web interface was developed using **Gradio** within the same Colab environment. The interface included an image upload component for leaf images, a text field to select the preferred language, and output sections to display the predicted disease and the generated explanation.

The image classification function and the LLM explanation function were linked together to form a complete workflow. The Gradio application was launched and tested using multiple sample images. This interface allows users, including farmers and gardeners, to interact with the system easily through a browser.

Step 6: Testing and Optimization

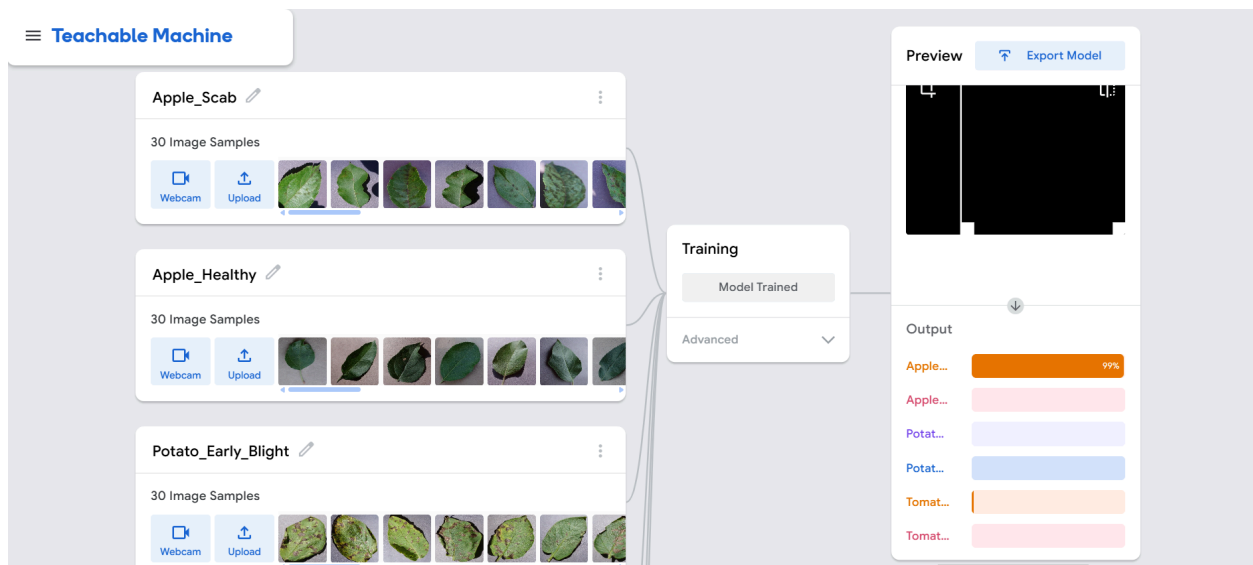
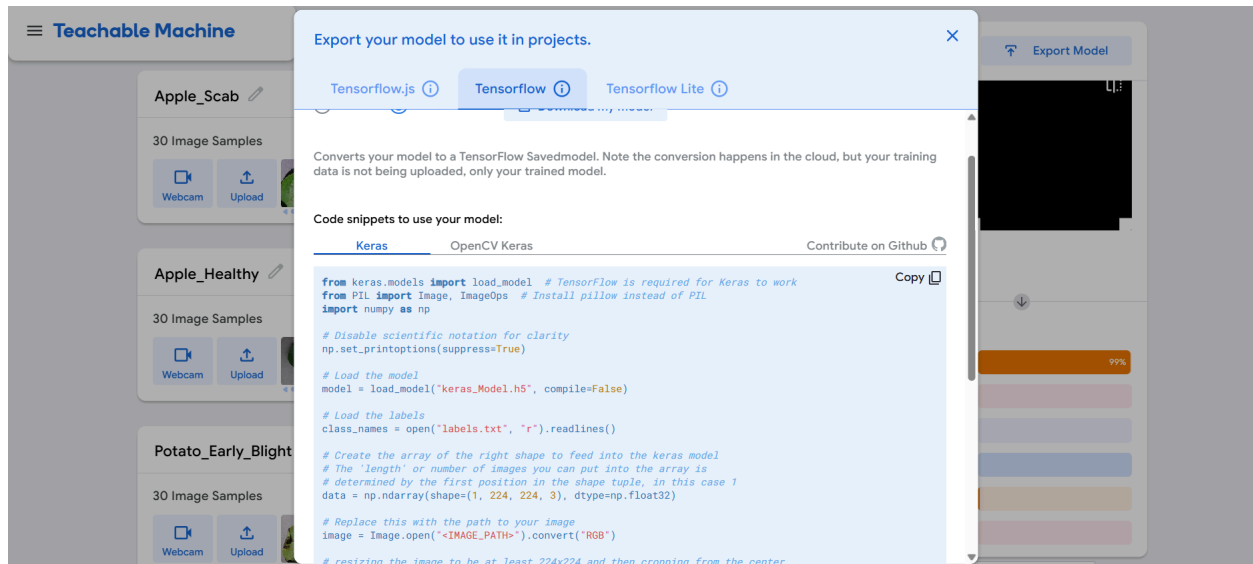
The complete system was tested using images from the Testing folder as well as external plant leaf images. Incorrect predictions were noted, and the model was refined when required by retraining with additional images or adjusting training parameters.

The quality of explanations was improved by refining the prompt structure and simplifying the language for better understanding by users with low technical literacy. The full workflow—from image upload to explanation display—was repeatedly tested through the Gradio interface to ensure consistency and reliability.

OUTPUTS/RESULTS:

1. Trained Model Output (Teachable Machine)

- The image classification model was successfully trained using Teachable Machine.
- The model was able to classify plant leaf images into six classes: Apple_Scab, Apple_Healthy, Potato_Early_Blight, Potato_Healthy, Tomato_Leaf_Mold, and Tomato_Healthy.
- During preview testing, the model showed correct predictions with high confidence for most images.



Model Prediction Results in Google Colab

- The trained TensorFlow model was successfully loaded in Google Colab.
- Test images from the testing dataset were passed to the model.
- The model correctly predicted the disease class for most input images.

- The predicted class label was extracted and passed to the LLM for explanation generation.

```
[ ] #@title Run this cell to Load Class labels

class_names = open("labels.txt", "r").readlines()
class_names = [name.split()[-1] for name in class_names]

print("Class labels loaded successfully!")
print(class_names)

... Class labels loaded successfully!
['Apple_Scab', 'Apple_Healthy', 'Potato_Early_Blight', 'Potato_Healthy', '...
```

```
#@title Upload a Plant Leaf Test image, provide its file name, then Run this cell

# "Upload a test image in Colab and provide its file name here (eg. Apple_scab_test
img_file = "Apple_scab_test_1.JPG" #@param {type:"string"}

# Prepare the input image
data = np.ndarray(shape=(1, 224, 224, 3), dtype=np.float32)

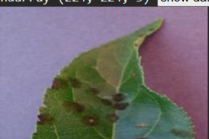
image = Image.open(img_file).convert("RGB")
size = (224, 224)
image = ImageOps.fit(image, size, Image.Resampling.LANCZOS)

image_array = np.asarray(image)
normalized_image_array = (image_array.astype(np.float32) / 127.5) - 1
data[0] = normalized_image_array

# Show image
image_array
```

ndarray (224, 224, 3) [show data](#)

img_file
Apple_scab_test_1.JPG
[Hide code](#)



3. LLM-Generated Disease Explanation and Treatment

- Based on the predicted disease, the Large Language Model generated a clear and farmer-friendly explanation.
- The output included:
 - Disease description
 - Key symptoms

- Immediate actions
 - Treatment suggestions
 - Preventive measures
- The explanation was generated in the user-selected language.

Plant: Apple **Status:** Unhealthy **Disease:** Apple Scab

... As an expert agricultural plant pathologist and crop-health advisor, I can confirm that Apple Scab is a common and often challenging fungal disease for apple growers. Let's break down this condition and how to manage it effectively.

A Brief Explanation of Apple Scab

Apple Scab is caused by the fungus *Venturia inaequalis*. It is one of the most widespread and damaging diseases of apples (and crabapples) in regions with cool, wet spring weather. The fungus overwinters primarily in fallen infected leaves on the orchard floor. In spring, when temperatures are mild and moisture is present, spores (ascospores) are released from these overwintered leaves and carried by wind to newly emerging leaves, flowers, and developing fruit. These primary infections lead to the characteristic scab lesions. From these primary lesions, secondary spores (conidia) are produced, which are then spread by splashing rain and wind, causing multiple cycles of infection throughout the growing season, especially during prolonged wet periods.

Key Symptoms

Apple Scab manifests distinct symptoms on various parts of the apple tree:

- **Leaves:** Small, olive-green to brownish, velvety spots appear, often first on the underside of young leaves. As lesions mature, they become darker, more defined, and can cause puckering, curling, or distortion of the leaf. Severely infected leaves may turn yellow and drop prematurely, weakening the tree.
- **Fruit:** Initially, small, dark, velvety spots appear, similar to those on leaves. As the fruit grows, these spots become scab-like, rough, corky, and may crack open, providing entry points for secondary rot organisms. Infected fruit can be misshapen, stunted, or drop prematurely. Scabs are often concentrated around the calyx (blossom) end of the fruit.
- **Twigs and Blossoms:** Less common but can occur. On twigs, lesions appear as small, raised, blister-like areas that later crack and become flaky. Blossom infection can cause darkening and dropping of flowers, reducing fruit set.

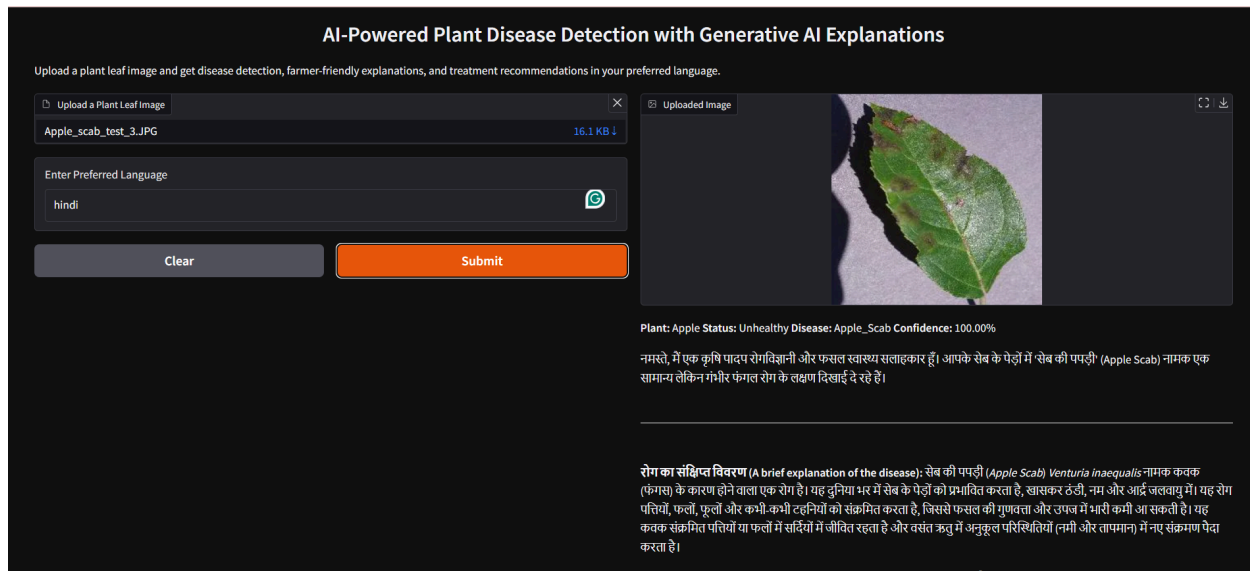
Immediate Actions a Farmer/Gardener Should Take

Upon identifying Apple Scab, immediate actions are crucial to limit its spread:

1. **Remove Infected Material:** Promptly remove and dispose of any fallen leaves, twigs, or fruit showing scab symptoms. Do not compost this material, as the fungus can survive and re-infect. Bag it and discard it or burn it (if local regulations allow).
2. **Sanitize Tools:** After pruning or working with infected trees, thoroughly clean and sanitize all pruning shears, saws, and other tools with a 10% bleach solution or rubbing alcohol to prevent spreading spores to healthy parts of the tree or other trees.

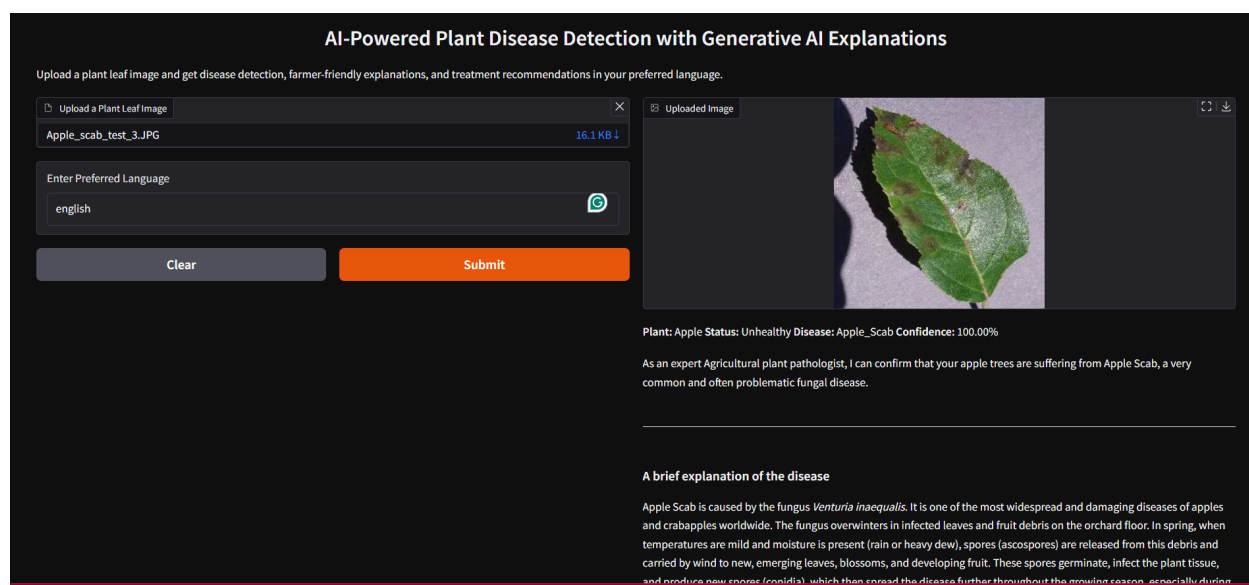
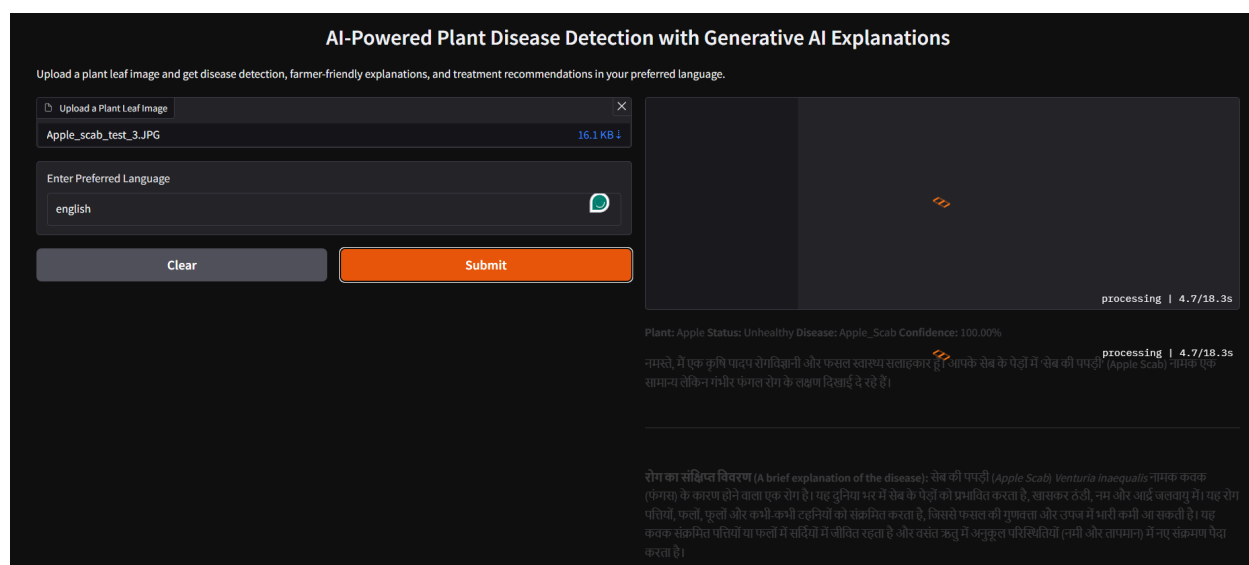
4. Multilingual Output Result

- The system supports multilingual responses.
- When a preferred language (e.g., Hindi/Tamil/etc.) was selected, the explanation was generated in that language.
- This feature improves accessibility for farmers from different regions.



5. Gradio Web Interface Output

- A Gradio-based web interface was successfully developed.
- Users can upload a leaf image, select a language, and receive disease predictions and explanations.
- The interface displays:
 - Uploaded image
 - Predicted disease status (Healthy/Unhealthy)
 - AI-generated explanation and treatment steps



LEARNINGS:

Through this project, I gained practical experience in building an end-to-end AI system that combines computer vision and generative AI. I learned how to prepare and organize image datasets, train a plant disease classification model using Teachable Machine, and export it as a TensorFlow SavedModel. I also learned how to integrate trained models into Google Colab, preprocess images, and run predictions. Additionally, I gained hands-on experience in using Large Language Models (Gemini/OpenAI/Claude) to generate structured, farmer-friendly explanations, treatments, and preventive measures. Building a Gradio



interface helped me understand how AI models can be deployed with simple user interfaces for real-world usage.

SPECIFIC CHALLENGES FACED:

Several challenges were encountered during the implementation of this project. Managing the dataset structure and ensuring correct class-label mapping was initially confusing. Loading the SavedModel correctly in Google Colab required careful handling of file paths and session resets. Integrating multiple components—image classification, LLM API calls, and Gradio UI—into a single workflow was challenging. Prompt design for generating clear and concise agricultural explanations also required multiple refinements. Additionally, limited API access and session timeouts in Colab sometimes interrupted progress.

FUTURE SCOPE:

The system can be further improved by expanding the dataset to include more plant species and disease types. Model accuracy can be enhanced by training with larger datasets and advanced deep learning architectures. Real-time image capture through mobile devices can be integrated for field use by farmers. Offline support and lightweight mobile deployment can improve accessibility in rural areas. Additional features such as weather-based disease prediction, fertilizer recommendations, and integration with government agricultural advisories can further enhance the usefulness of the system.

SUMMARY AND CONCLUSIONS:

This project successfully demonstrates an AI-powered plant disease detection system that combines image classification and generative AI-based explanations. Using Teachable Machine for model training, TensorFlow for inference, and Large Language Models for explanation generation, the system provides accurate disease detection along with practical treatment guidance. The Gradio interface enables easy interaction, making the solution user-friendly and accessible. Overall, the project highlights how AI can support farmers by simplifying disease diagnosis and promoting better crop health management.

REFERENCES:

Teachable Machine – <https://teachablemachine.withgoogle.com>

Google Colab – <https://colab.research.google.com>

TensorFlow Documentation – <https://www.tensorflow.org>

Gradio Documentation – <https://gradio.app>

Google Gemini API Documentation – <https://ai.google.dev/gemini-api/docs>

Plant Disease Dataset (Provided by Instructor / Kaggle Source)